

Fig. 7: Connection of the seven-segment display with Raspberry Pi

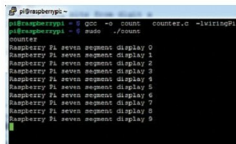


Fig. 8: Counter display in seven-segment display

TABLE II
Connection between the
Seven-segment Display
and Raspberry Pi

Seven-segment display pins	Raspberry Pi pins
CA	Pin 2
a	Pin 3
b	Pin 5
c	Pin 7
d	Pin 11
e	Pin 13
f	Pin 15
g	Pin 19
dp	Pin 21

the WiringPi system and assumes that the calling program is going to use the wiringPi pin numbering scheme. This function must be called at the start of your program.

void pinMode (int pin, int mode).

The pin number is obtained from the pins table. This sets the mode of a pin to either INPUT, OUTPUT or PWM_OUTPUT.

void digitalWrite (int pin, int value). Writes the value HIGH or LOW (1 or 0) to the given pin which must have been previously set as an output using pinMode() function.

void delay (unsigned int howLong). This causes program execution to pause for at least howLong milliseconds. Due to the multi-tasking nature of Linux, it could be longer. Note that the maximum delay is an unsigned 32-bit integer or approximately 49 days.

Project 2: Seven-segment counter display using Wiring Pi C

In this project, we will display the numbers from 0 to 9 in seven-segment display using Wiring Pi C.

Circuit

Connect the seven-segment display (common anode) to the pins of Raspberry Pi as shown in Fig. 7. Connect the seven-segment display to the Raspberry Pi pins as shown in Table II.

Note. Add 330- or 470-ohm resistors for each connection (not shown in the Fig. 7 to avoid complexity).

Software

Open LxTerminal and create a new C file using following command:

```
$ touch counter.c
```

Start nano text editor as:

```
$ sudo nano counter.c
```

Type below C code or paste it to the LxTerminal.

```
-----counter.C-----
#include<stdio.h>
#include<wiringPi.h>

//GPIO to seven-segment pins
a,b,c,d,e,f,g,dp

//these are wiring Pi pin numbers
unsigned int
gpio[8]={8,9,7,0,2,3,12,13};

unsigned int b[8]; // for bit values
of one seven-segment digit

//seven-segment code for 0 to 9
digits
unsigned int ssc[10]={0xc0,0xf8,0xa4,
0xb0,0x99,0x92,
```

```
0x82,0xf8,0x80,0x90};
```

```
void seg_count(int);
int main()
{
    int i,b;
    printf("counter\n");
    if(wiringPiSetup () == -1)
        //initialisation
        return 1;
```

```
for(i=0;i<=7;i++)
pinMode (gpio[i], OUTPUT);
// pin mode set to output
```

```
while(1)
```

```
{
    for(i=0;i<=9;i++)
```

```
{
    seg_count(i);
    delay(1000);
}
```

```
}
return 0;
```

```
void seg_count (int x)
```

```
{
    int i;
    //separating bits from digit x
    for(i=0;i<=7;i++)
        b[i]=(ssc[x]>>i)& 0x01;
```

```
printf ("Raspberry Pi seven segment
display %d\n",x);
}
```

```
for (i=0;i<=7;i++) // setting
GPIO pins as per seven seg code
digitalWrite(gpio[i],b[i]);
}
```

Once you are finished, press Ctrl + O to save and then Ctrl + X to exit from the nano editor.

Now compile the code:

```
$ gcc -o counter counter.c -lwiringPi
and then execute it as shown in Fig. 8.
$ sudo ./count
```

You will see different numbers displayed in a sequence and the terminal window will display messages as shown in Fig. 8. You can abort the execution by pressing Ctrl+Z. ●

The author is pursuing M.Tech in electronics at Walchand College of Engineering, Sangli, Maharashtra